

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 1 | AULA 02 -
**EXCEÇÕES E
MANIPULAÇÃO DE
ERROS**

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	24
REFERÊNCIAS.....	25

EMENDAS

O QUE VEM POR AÍ?

Nesta aula aprenderemos a tratar erros de programas que ocorrem em tempo de execução. Vamos entender a diferença entre erros de compilação e exceção, os principais tipos de exceções em Java, como capturar e tratar exceções, como criar e usar nossas próprias exceções e as boas práticas de programação com exceções.

Assim, esperamos que o aluno e aluna possam aplicar de forma correta o uso de exceções, deixando o código mais organizado também na tratativa de erros.

HANDS ON

Iniciaremos a aula explicando o que são exceções e o que acontece quando ocorre uma. Em seguida, iremos demonstrar como e quando tratar exceções.

Explicaremos os principais tipos de exceções e como e por quais motivos devemos criar nossas próprias exceções. Por fim, iremos explicar boas práticas de programação com exceções.



SAIBA MAIS

O QUE SÃO EXCEÇÕES NA PROGRAMAÇÃO

Imagine o seguinte cenário: precisamos desenvolver um sistema que deve acessar dados armazenados em arquivos no disco do servidor. Durante o desenvolvimento do sistema teremos alguns erros de compilação que a pessoa desenvolvedora resolverá no decorrer da construção do sistema.

Esse processo é normal, até chegarmos na etapa em que precisamos acessar os arquivos no disco. Durante os testes, a pessoa desenvolvedora deixou os arquivos no diretório esperado e tudo funcionou bem. Mas ao disponibilizar o sistema em produção, ele não funcionou. A causa: os diretórios onde estão os arquivos têm nomes diferentes do que a aplicação esperava. Na programação de sistemas, isso é uma exceção.

Outro exemplo: desenvolvemos um sistema de contabilidade que recebe diversos dados informados pelo usuário. Com esses dados, fizemos diversos cálculos antes de salvar no banco de dados. Tudo funciona bem, até que o usuário resolve informar um número 0 (zero) e o sistema falha. A causa: um dos cálculos do sistema usa esse número em uma divisão e, assim, aconteceu uma divisão por zero. Isso resulta em uma exceção. Outra possibilidade é que na hora de salvar os dados no banco de dados, o sistema de banco de dados está fora (desligado ou inacessível), o que também é uma exceção.

Como vimos, as exceções podem acontecer em qualquer parte de um sistema, por diversas razões. Pode faltar espaço em disco ou memória RAM para o sistema, arquivos podem estar corrompidos, a rede pode estar indisponível etc.

Podemos então considerar que exceções são erros que acontecem em tempo de execução do sistema, diferente dos erros de compilação, que ocorrem durante o tempo de codificação.

Nós, como pessoas programadoras de sistemas, precisamos estar cientes desses problemas de exceção, prevendo onde eles podem acontecer e, assim, trazer tratativas dessas exceções, fazendo fluxos alternativos, salvando a causa do erro em arquivos de log e/ou informando amigavelmente o problema ocorrido para o usuário.

O que acontece quando ocorre uma exceção

Antes de explicar o que acontece quando ocorre uma exceção, precisamos esclarecer como funciona a “pilha” de chamadas em um sistema. Normalmente um sistema é composto por diversos métodos ou funções que estão dentro de classes ou arquivos. Em um fluxo sistêmico, tudo começa na origem que, normalmente, é um método ou função. Essa função ou método chama outras sucessivamente até que o fluxo chegue em seu destino.

Neste fluxo, a origem pode ser um método que obtém dados do usuário, o meio do fluxo pode ser processar esses dados, adicionando informações vindas de fontes externas (por exemplo APIs), e o fim do fluxo pode ser salvar esses dados no banco de dados.

Em um fluxo síncrono, como é a maior parte dos fluxos de um sistema, a origem fica esperando a resposta da camada intermediária, que por sua vez espera a camada final. Se uma dessas camadas demorar ou travar, todos ficarão travados esperando o retorno.

Todas essas chamadas de métodos ou funções são o que chamamos de “pilha de chamadas” (“call stack”, em inglês). A figura 1 ilustra uma pilha de chamadas.

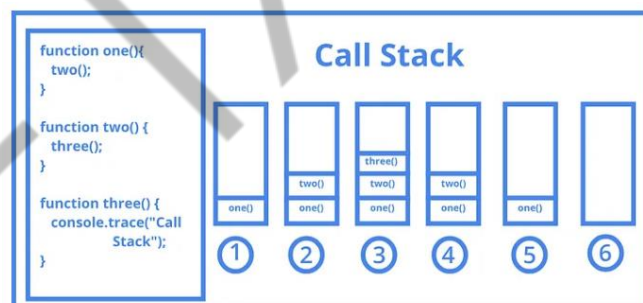


Figura 1 – Pilha de Chamadas
Fonte: The Startup (2019)

Agora que aprendemos como funciona um fluxo sistêmico, vamos entender como se comporta uma exceção. O diagrama da figura 2 ilustra uma jornada sistêmica em que um usuário inicia o fluxo. Em seguida, mais dados são obtidos por uma API externa e, por fim, ela é salva no banco. As setas de retorno representam o retorno do método (mesmo sendo void).

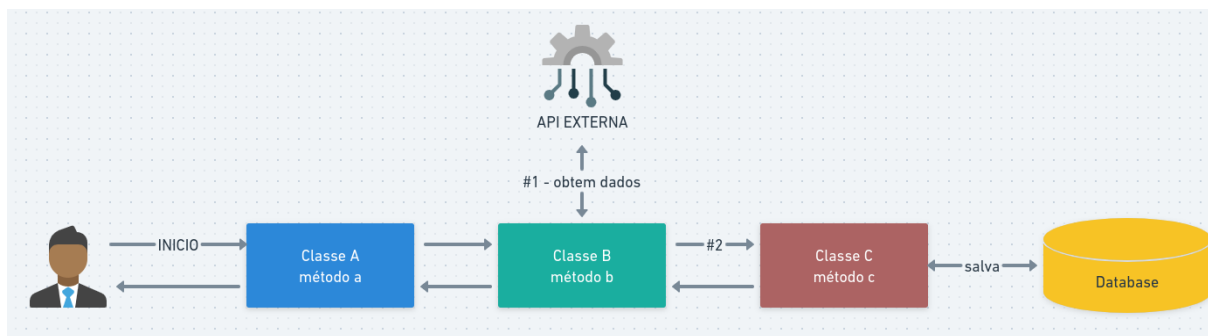


Figura 2 – Fluxo de sistema entre classes e seus métodos
Fonte: elaborado pelo autor (2024)

Supondo que nenhum desses métodos tratem exceção (veremos mais adiante como fazer isso), se em algum desses métodos ela acontecer, será lançada para o método que a chamou. Por exemplo: se o método “c” falhar em salvar no banco de dados, a exceção será lançada para quem a chamou, no caso o método “b”, e o “b” lançará essa mesma exceção para quem o chamou, no caso o “a”. O “a”, então, irá lançar essa mesma exceção para quem o chamou, no caso o usuário.

Esse é um exemplo em que o usuário veria aquele erro não tratado em sua tela. O ideal seria ao menos o método “a” tratar a exceção para melhorar a mensagem de erro na tela. A figura 3 demonstra o lançamento da exceção nesse exemplo.

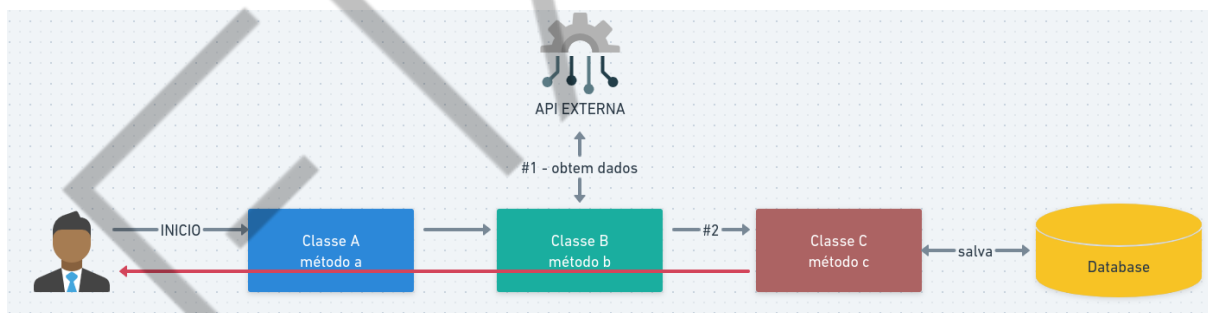


Figura 3 - Exceção passa por todos os elementos da pilha de chamadas
Fonte: elaborado pelo autor (2024)

Quando e como tratar exceções

É comum pessoas desenvolvedoras terem dúvidas sobre quando devem tratar as exceções. Para ajudar a resolver isso, vamos a um exemplo.

Imagine um fluxo sistêmico em que se o acesso ao database falhar, o sistema deve notificar o administrador e tentar salvar os dados em outro banco de dados. Esse é um caso que devemos tratar a exceção para que possamos fazer um fluxo alternativo. O fluxo do nosso sistema deve ficar conforme a figura 4.

Exceções e Manipulação de Erros

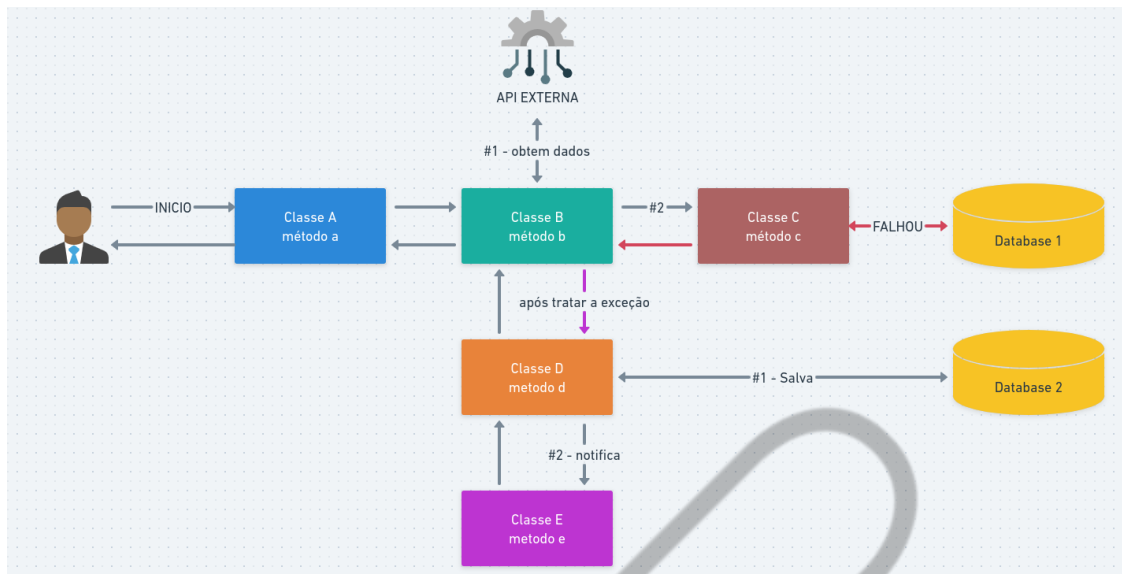


Figura 4 – Método b tratando exceção vinda do método c
Fonte: elaborado pelo autor (2024)

Agora vamos descobrir como tratar essa exceção no código. Em Java, utilizamos o comando “try/catch” para capturar e tratar a exceção. O código 1 demonstra a classe B antes e depois de tratá-la.

```
public class ClasseB {  
  
    private ClasseC classec;  
  
    public void metodoB(String[] param) {  
        classec.metodoC(param);  
    }  
}
```

```
public class ClasseB {  
    private ClasseC classec;  
    private ClasseD classed;  
  
    public void metodoB(String[] param) {  
        try {  
            classec.metodoC(param);  
        } catch (Exception e) {  
            classed.metodoD(param);  
        }  
    }  
}
```



```
}  
}
```

Código 1 – Classe antes e depois de tratar exceção
Fonte: elaborado pelo autor (2024)

Tipos de exceções

Cada plataforma de desenvolvimento possui seus conjuntos de ferramentas e tipos de exceções. Neste tópico, nos aprofundaremos nos tipos de exceções em Java.

Java é uma linguagem de programação totalmente orientada a objetos. Por isso, qualquer coisa nela é representada por classes (tipos); as exceções também.

Uma parte das classes de exceções existente é do próprio SDK. Visto que os sistemas em Java rodam na máquina virtual Java (JVM), se ela falhar, lançará uma exceção correspondente a essa falha. Um exemplo é a “OutOfMemoryError”, um problema relacionado à falta de memória ao executar um programa.

Existe uma hierarquia (herança) de classes de exceções em Java. O objetivo é separar em categorias e hierarquias essas exceções a fim de facilitar sua organização. Afinal, são inúmeras as exceções que podem ocorrer em um sistema.

A figura 5 traz a lista de hierarquia das classes de exceções do Java.

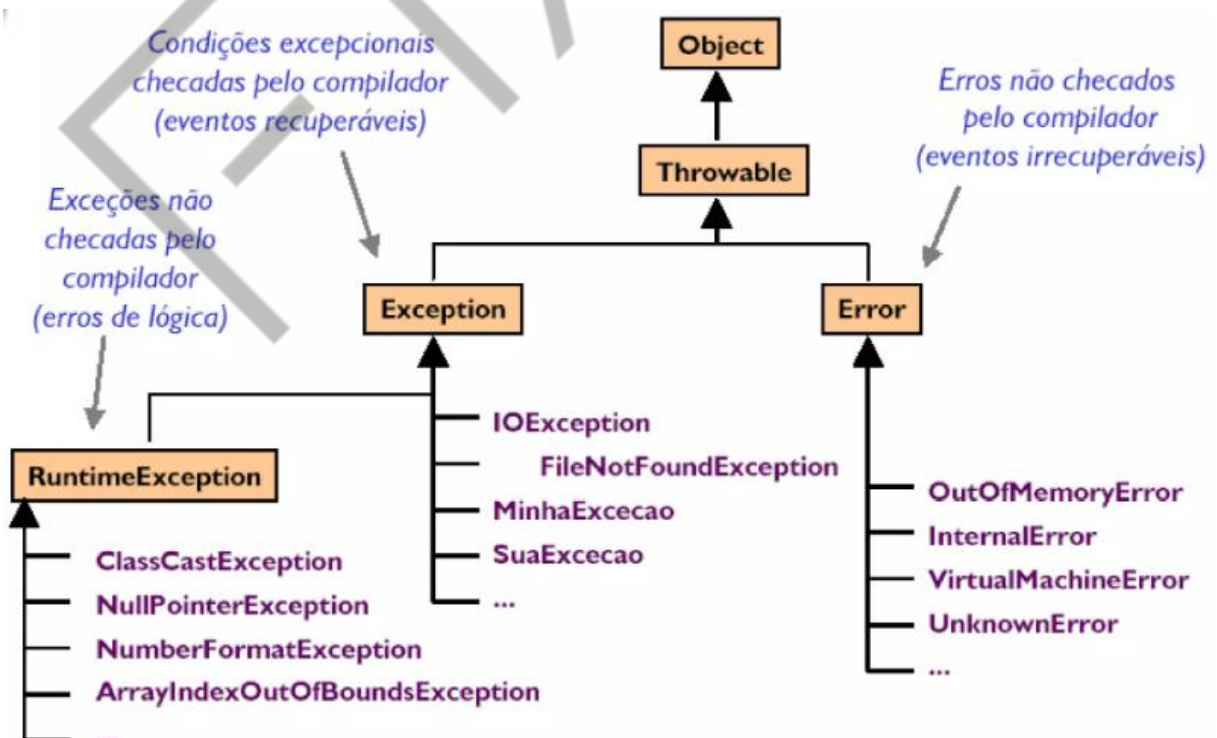


Figura 5 – Hierarquia de exceções em Java
Fonte: Regis Pires Magalhães (s.d.)

Focaremos aqui nas seguintes exceções: “Exception” e “RuntimeException”.

Exception

A classe “Exception” representa as exceções “cheçadas” pela JVM. Significa que se um método lança essa exceção (ou qualquer uma de suas subclasses), o método chamador obrigatoriamente deve tratar essa exceção ou deve relançar ela, explicitando na assinatura do método esse lançamento. Caso isso não seja respeitado, ocorrerá um erro de compilação.

O código 2 demonstra uma classe com um método que chama outro método que, por sua vez, lança uma Exception.

Todos os métodos da ClasseA chamam o mesmo método da ClasseB. O método `metodoSemTratarSemLancar` apresenta erro de compilação, pois ele faz a chamada do método da ClasseB sem empregar qualquer tratamento de exceção. O método `metodoRelancando` não deu erro de compilação, pois apesar de não tratar a exceção proveniente do método que ele chama, ele deixa claro em sua assinatura que lança essa exceção; portanto, nesse caso ele repassa a exceção para quem chamou. Finalmente o método `metodoTratando` faz a tratativa da exceção e por isso não ocorre erro de compilação.

```
public class ClasseA {  
  
    private ClasseB classeB;  
  
    public void metodoSemTratarSemLancar() {  
        classeB.metodoQueLancaException();//Erro Comp.  
    }  
  
    public void metodoRelancando() throws Exception {  
        classeB.metodoQueLancaException();  
    }  
}
```

```
public void metodoTratando() {  
    try {  
        classeB.metodoQueLancaException();  
    } catch (Exception e) {  
        //Faz alguma coisa  
    }  
}  
}
```

Código 2 – Método que chama outro que lança Exception
Fonte: elaborado pelo autor (2024)

Por enquanto não iremos mostrar a ClasseB (que lança a exception) para mantermos o foco na tratativa das exceções.

RuntimeException

A classe "RuntimeException" representa as exceções “não checadas” pela JVM. Significa que se um método lança essa exceção ou qualquer uma de suas subclasses, o método chamador **não precisa** tratar essa exceção nem explicitar na assinatura do método esse lançamento. Nenhum erro de compilação acontecerá.

Mudamos as chamadas do método da ClasseA para chamar um método que não lança a Exception, mas sim a RuntimeException. Vejamos como ficou o código 3. A única mudança foi que agora chamamos um método (da ClasseB) que lança RuntimeException. Notem que não temos mais erros de compilação. O método “metodoRelancando” não precisa do “throws” em sua assinatura, que ficou opcional, e a tratativa da exceção no método “metodoTratando” continua funcionando, afinal RuntimeException é subclasse de Exception.

```
public class ClasseA {  
  
    private ClasseB classeB;  
  
    public void metodoSemTratarSemLancar() {
```

```
    classeB.metodoQueLancaRuntimeException();  
}  
  
public void metodoRelancando() throws Exception {  
    classeB.metodoQueLancaRuntimeException();  
}  
  
public void metodoTratando() {  
    try {  
        classeB.metodoQueLancaRuntimeException();  
    } catch (Exception e) {  
        //Faz alguma coisa  
    }  
}  
}
```

Código 3 - Métodos que chamam outros métodos que lançam RuntimeException
Fonte: elaborado pelo autor (2024)

Lançamento de exceptions

Como mencionamos, as exceções são lançadas sempre que ocorrem situações adversas em um fluxo sistêmico. Então, por exemplo, quando temos que salvar dados no banco de dados utilizando JDBC (API padrão do Java para acesso a banco de dados) essa API pode lançar uma exception. Cabe a nós a decidir o que fazer ou como tratá-la.

No código 4 temos um exemplo de uso do JDBC. O método “getConnecion” da classe DriverManager lança uma SQLException (subclasse de Exception), então somos obrigados a tratá-la com o uso de try/catch ou relançar usando throws na assinatura do método.

Reforçamos que a classe “DriverManager” é uma classe do JDBC (core do Java) e não podemos mudá-la, então temos que aceitar a situação de lidar com a SQLException.

```
public class ConexaoDatabase {
```

```
public void conecta() {  
    String url = "jdbc:mysql://localhost/anyDatabase";  
    String usuario = "any-user";  
    String senha = "any-pass";  
  
    try {  
        Connection conexao =  
            DriverManager.getConnection(url, usuario, senha);  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

Código 4 – Exemplo de JDBC
Fonte: elaborado pelo autor (2024)

Mas em nossos códigos podemos também lançar as exceções sempre que acharmos necessário. Lembrando que se lançarmos `Exception`, forçaremos os chamadores a lidar com ela, mas se for uma `RuntimeException` não há essa obrigação.

O código 5 demonstra um caso em que lançamos uma `RuntimeException`. Note que dada uma condição, lançamos a exceção. Neste caso é uma exceção de negócio, pois trata uma regra. Para lançar a exceção usamos a palavra reservada `throw` seguido de `new` e a classe da exceção que queremos lançar.

Se mudássemos de `RuntimeException` para `Exception`, teríamos que adicionar `throws Exception` na assinatura do método.

É importante notar que não é uma boa prática de programação lançarmos exceções genéricas. O ideal seria criarmos nossa própria classe de exceção e lançá-la. Veremos como fazer isso.

```
public class AlunoService {  
  
    private AlunoDatabaseGateway alunoDatabaseGateway;
```

```
public void salvar(Aluno aluno) {  
  
    if(aluno.estaMatriculado()) {  
        alunoDatabaseGateway.salvar(aluno);  
    } else {  
        throw new RuntimeException("Aluno não  
está devidamente matriculado");  
    }  
}  
}
```

Código 5 – Exemplo de como e onde lançar exceções
Fonte: elaborado pelo autor (2024)

Criando nossas classes de exceções

Conforme mencionado, a boa prática diz para não lançarmos exceções genéricas. Se usarmos exceções genéricas podemos ter problema em identificar a causa do erro e, em algumas situações, precisamos tratar um erro específico e não qualquer um.

Imagine a seguinte situação: dado um cadastro de alunos, o sistema deve, além de salvar os dados na base, notificar via SMS o usuário. Se falhar ao salvar no banco, o sistema deve exibir uma mensagem amigável da falha. Porém, se falhar o SMS, deve avisar que o cadastro teve sucesso parcial.

O código 6 ilustra esse problema. Na linha “alunoService.salvar(novoAluno)” podem ocorrer exceções em diferentes momentos. Duas em especial nos interessam: erro no database e erro na notificação. Se usarmos exceções genéricas do tipo Exception ou RuntimeException como saberemos qual delas aconteceu? Vamos ver o resultado no código 7.

```
public class AlunoController {  
  
    private AlunoService alunoService;
```

```
public void cadastrar(Aluno novoAluno) {  
  
    try {  
        alunoService.salvar(novoAluno);  
  
    } catch (Exception e) {  
        //Se der erro de  
        //database deve exibir erro amigável  
  
        //Se der erro de notificação deve  
        //informar que processou parcialmente  
    }  
}  
}
```

Código 6 — Exemplo para problema de cadastro
Fonte: elaborado pelo autor (2024)

O código 7 demonstra a possibilidade de capturar e tratar diferentes tipos de exceções dentro de cada “catch”. As classes de exceções “ErroAoAcessarDatabaseException” e “ErroAoNotificarException” foram criadas e mantidas no sistema. Adiante, temos a demonstração delas. Já no último “catch”, capturamos qualquer exceção que seja diferente das outras duas anteriores.

```
public class AlunoController {  
  
    private AlunoService alunoService;  
  
    public void cadastrar(Aluno novoAluno) {  
  
        try {  
            alunoService.salvar(novoAluno);  
  
        } catch (ErrorAoAcessarDatabaseException e) {  
            exibirErroDatabase();  
        } catch (ErroAoNotificarException e) {
```

Exceções e Manipulação de Erros

```
exibirErroNotificacao();  
} catch (Exception e) {  
    exibirErroGenerico();  
}  
}  
  
private void exibirErroGenerico() {  
    // TODO Auto-generated method stub  
  
}  
  
private void exibirErroNotificacao() {  
    // TODO Auto-generated method stub  
  
}  
  
private void exibirErroDatabase() {  
    // TODO Auto-generated method stub  
  
}  
}
```

Código 7 — Capturar e tratar diferentes tipos de exceções
Fonte: elaborado pelo autor (2024)

Os códigos 8 e 9 demonstram as exceções que criamos.

```
public class ErrorAoAcessarDatabaseException extends RuntimeException {  
  
}
```

Código 8 — Exceções criadas (1)
Fonte: elaborado pelo autor (2024)

```
public class ErroAoNotificarException extends RuntimeException {  
  
}
```

Código 9 — Exceções criadas (2)
Fonte: elaborado pelo autor (2024)

Comando finally

O “finally” é uma instrução importante que pode ser utilizada junto com o “try”. Ele executa sempre o código que está dentro do bloco que o compreende. Vamos para mais um exemplo.

Suponha que após tentar salvar os dados do aluno, precisamos que um componente da tela seja resetado. Esse componente **sempre** precisa ser resetado, independente de sucesso ou falha. Para isso, usaremos o finally.

O código 10 demonstra seu uso. O que deve ficar claro é que o método contido no bloco “finally” será chamado **independentemente** do que ocorrer entre as linhas dentro do “try”. Ou seja, se o método “salvar” (dentro do “try”) for chamado com sucesso, o método “resetaComponente” será chamado. Se “salvar” falhar, ele vai entrar no “catch” e em seguida chamar o “finally”.

```
public class AlunoController {  
  
    private AlunoService alunoService;  
  
    public void cadastrar(Aluno novoAluno) {  
  
        try {  
            alunoService.salvar(novoAluno);  
  
        } catch (ErrorAoAcessarDatabaseException e) {  
            exibirErroDatabase();  
  
        } catch (ErroAoNotificarException e) {  
            exibirErroNotificacao();  
  
        } catch (Exception e) {  
            exibirErroGenerico();  
  
        } finally {  
            resetaComponente();  
        }  
    }  
}
```

Exceções e Manipulação de Erros

```
private void resetaComponente() {  
    // TODO Auto-generated method stub  
  
}  
  
private void exibirErroGenerico() {  
    // TODO Auto-generated method stub  
  
}  
  
private void exibirErroNotificacao() {  
    // TODO Auto-generated method stub  
  
}  
  
private void exibirErroDatabase() {  
    // TODO Auto-generated method stub  
  
}  
}
```

Código 10 – Uso de finally
Fonte: elaborado pelo autor (2024)

Reforçando que não é obrigatório o uso de “catch” para usar o finally. Veja o código 11. O código dentro do “finally” será chamado independentemente do que acontecer dentro do “try”.

```
public class AlunoController {  
  
    private AlunoService alunoService;  
  
    public void cadastrar(Aluno novoAluno) {  
  
        try {  
            alunoService.salvar(novoAluno);  
        }  
    }  
}
```

```
    } finally {  
        resetaComponente();  
    }  
}  
  
private void resetaComponente() {} //Logica não implementada  
  
private void exibirErroGenerico() {} //Logica não implementada  
  
private void exibirErroNotificacao() {} //Logica não implementada  
  
private void exibirErroDatabase() {} //Logica não implementada  
}
```

Código 11 — Não é obrigatório o uso de “catch” para usar o finally
Fonte: elaborado pelo autor (2024)

BOAS PRÁTICAS USANDO EXCEÇÕES

Listaremos aqui alguns casos de boas práticas de uso de exceções, algumas das quais já abordamos no decorrer da aula.

Criar sempre as exceções do projeto

Conforme explicado anteriormente, sempre que for necessário lançar uma exceção, não use uma do Java, crie a sua. Normalmente as exceções do sistema ficam em um pacote “exception” do projeto.

Utilize logs, mas com cautela

É muito importante logar alguns dados do sistema. Eles são necessários para investigação de bugs e erros que ocorrem no servidor. Ao mesmo tempo, temos que ter cuidado com os logs, pois se aplicados em excesso podem atrapalhar a investigação e os custos de cloud.

Em relação ao log de exceções, tenha atenção ao colocar logs na cláusula “catch” e, se possível, com os dados de input do método. Veja um exemplo no código 12, observe o log dentro do “catch”.

```
@Slf4j  
public class AlunoController {
```

Exceções e Manipulação de Erros

```
private AlunoService alunoService;

public void cadastrar(Aluno novoAluno) {

    try {
        alunoService.salvar(novoAluno);

    } catch (Exception e) {
        log.error(e.getMessage(), e);
        exibirErroGenerico();

    } finally {
        resetaComponente();
    }
}

private void resetaComponente() {} //Logica não implementada

private void exibirErroGenerico() {} //Logica não implementada
}
```

Código 12 — Log de exceções
Fonte: elaborado pelo autor (2024)

A dica é colocar logs antes de lançar qualquer exceção. Isso ajuda muito a entender onde e qual é a causa de um interrompimento do fluxo sistêmico. No código 13 temos um exemplo de logs adicionados antes de lançar a exceção.

```
@Slf4j
public class AlunoService {
    private AlunoDatabaseGateway alunoDatabaseGateway;
    private NotificacaoGateway notificacaoGateway;

    public void salvar(Aluno aluno) {

        if(!aluno.estaMatriculado()) {
            log.info("aluno={}", aluno);
            log.warn("Aluno sem Matricula");
            throw new AlunoSemMatriculaException();
        }
    }
}
```

```
}  
alunoDatabaseGateway.salvar(aluno);  
notificacaoGateway.sms();  
}  
}
```

Código 13 — Logs adicionados antes de lançar a exceção
Fonte: elaborado pelo autor (2024)

Não faça "catch" sem código

Nunca adicione try/catch sem código dentro do "catch". Se isso for necessário, pois não há nenhuma ação a ser feita, ao menos coloque algum log. Se mesmo assim não for necessário um log, tenha certeza da necessidade de uso do "catch".

No código 14, temos um exemplo de "catch" sem código. A questão aqui é que, se acontecer um problema, o fluxo não será interrompido. Isso pode até ser necessário, depende da regra, mas em uma investigação de bugs podemos pensar que o fluxo executou com sucesso, o que não ocorreu, já que uma exceção pode ter acontecido ao "salvar". Então, é recomendado no mínimo um log de warning dentro do catch.

```
public class AlunoService {  
  
    private AlunoDatabaseGateway alunoDatabaseGateway;  
    private NotificacaoGateway notificacaoGateway;  
  
    public void salvar(Aluno aluno) {  
  
        try {  
            alunoDatabaseGateway.salvar(aluno);  
  
        } catch (Exception e) {  
            //FIXME: Não faça isso :(  
        }  
        notificacaoGateway.sms();  
    }  
}
```

Código 14 – Evitar catch sem código
Fonte: elaborado pelo autor (2024)

Não faça retorno de “código de erro” ou “true”/“false”

Se um método precisa reportar que sua operação foi um sucesso ou falha não o faça retornar códigos, como 1 é sucesso e 0 é erro, ou booleanos, do tipo true é sucesso e false é falha. Em caso de falha, em vez de retornar códigos ou booleanos lance exceções.

O código 15 demonstra o que **não** deve ser feito, pois os “return” desse método não deveriam existir.

```
@Slf4j
public class AlunoService {
    private AlunoDatabaseGateway alunoDatabaseGateway;
    private NotificacaoGateway notificacaoGateway;

    public int salvar(Aluno aluno) {
        if(!aluno.estaMatriculado()) {
            log.info("aluno={}", aluno);
            log.warn("Aluno sem Matricula");
            //throw new AlunoSemMatriculaException();
            return 0; //FIXME
        }
        alunoDatabaseGateway.salvar(aluno);
        notificacaoGateway.sms();
        return 1; //FIXME
    }
}
```

Código 15 — Não utilize código de sucesso ou falha
Fonte: elaborado pelo autor (2024)

Em vez de retornar código, faça lançar exceções, conforme o código 16.

```
@Slf4j
public class AlunoService {
    private AlunoDatabaseGateway alunoDatabaseGateway;
    private NotificacaoGateway notificacaoGateway;

    public void salvar(Aluno aluno) {

        if(!aluno.estaMatriculado()) {
```

Exceções e Manipulação de Erros

```
log.info("aluno={}", aluno);  
log.warn("Aluno sem Matricula");  
throw new AlunoSemMatriculaException();  
}  
alunoDatabaseGateway.salvar(aluno);  
notificacaoGateway.sms();  
}  
}
```

Código 16 – Lance exceções, nunca código de erro
Fonte: elaborado pelo autor (2024)

O QUE VOCÊ VIU NESTA AULA?

Nessa aula aprendemos o que são exceções em um sistema de software, como elas ocorrem e como e quando devemos tratá-las. Aprendemos a criar nossas próprias exceções e utilizá-las. Por fim, vimos boas práticas de programação usando exceções.

EMEND

REFERÊNCIAS

CARDOSO, C. **Orientação a Objetos na Prática**. São Paulo: Moderna Ciência, 2020.

MARTIN, R. **Código Limpo**. Rio de Janeiro: Alta Books, 2011.

MARTIN, R. **Arquitetura Limpa**. Rio de Janeiro: Alta Books, 2018.

ERRO

PALAVRAS-CHAVE

Exceções em Java. Try. Catch. Finally.

EXEMPLO



POSTECH